`:status` parameter provides the value used in the status header in the HTTP response. It defaults to `"200 OK"`. Do not use `render` with a 3xx status to do redirects; Rails has a `redirect` method for this purpose.

The `:layout` parameter determines whether the result of the rendering will be wrapped by a layout. (We first came across layouts in *Iteration C2: Adding a Page Layout*. We'll look at them in depth starting in *Reducing Maintenance with Layouts and Partials*.) If the parameter is `false`, no layout will be applied. If set to `nil` or `true`, a layout will be applied only if there's one associated with the current action. If the `:layout` parameter has a string as a value, it'll be taken as the name of the layout to use when rendering. A layout is never applied when the `:nothing` option is in effect.

The `:content_type` parameter lets you specify a value that will be passed to the browser in the Content-Type HTTP header.

Sometimes it's useful to be able to capture what would otherwise be sent to the browser in a string. The `render_to_string` method takes the same parameters as `render` but returns the result of rendering as a string—the rendering is not stored in the response object and so won't be sent to the user unless you take some additional steps.

Calling `render_to_string` doesn't count as a real render. You can invoke the real `render` method later without getting a `DoubleRender` error.

Sending Files and Other Data

We've looked at rendering templates and sending strings in the controller. The third type of response is to send data (typically, but not necessarily, file contents) to the client.

```
send_data(data, options…)
```

This sends a data stream to the client. Typically the browser will use a combination of the content type and the disposition, both set in the options, to determine what to do with this data.